MODULE

Introduction à l’informatique

Objectif, progression et organisation du module

1re année géomatique

2027-2028

Objectif du module

Consolider ou acquérir les bases informatiques nécessaires aux autres enseignements, en particulier ceux de programmation et les TD utilisant Python.

Demandé dans les autres cours	Vu dans ce module
« ouvrez le projet fourni »	éditeur de code, arborescence d'un projet (séances 1 et 2)
« lancez le script sur vos données »	ligne de commande, chemins, arguments d'un programme (séances 1 et 3)
« installez Python et numpy »	environnement, et fichier qui le décrit pour un autre poste (séance 5)
« rendez votre code »	git, README, dépôt sur GitHub (séances 2, 3 et 6)

Objectifs du module liés à la programmation

Maîtriser la forme d'un projet de code : documentation, organisation des fichiers, bibliothèques et environnements.

	ce cours	cours de programmation
Quel algorithme choisir ?		oui
Comment écrire cette boucle ?		oui
Où mettre ce fichier ?	oui	
Comment lancer le script ailleurs ?	oui	

Progression du module

Sept séances de deux heures. Chaque séance alterne explications courtes et TD sur machine.

Utiliser	Développer	Livrer
des programmes et des fichiers	un programme, seul puis à plusieurs	un programme à une autre personne
Séance 1 · Logiciels, fichiers, terminal et premier programme ordinateur et logiciels ; fichiers et chemins ; ligne de commande ; programme Python et notebook	**Séance 2 · Éditeur de code et git local** VS Code ; versions, branches et fusion avec git	**Séance 5 · Réutiliser des programmes et des bibliothèques** subprocess et PATH ; bibliothèques, dépendances, environnements
	Séance 3 · Fichiers, chemins et programme en ligne de commande lecture et écriture de fichiers ; chemins et arguments : pathlib, argparse	**Séance 6 · Forge, mots de passe et secrets** GitHub : clone, push, pull ; authentification ; secrets
	Séance 4 · Projet d'application 1 un script écrit pas à pas, un commit par étape	**Séance 7 · Projet d'application 2** une branche fusionnée par une pull request

En brun, les deux TD d’application des séances précédentes, avec un travail à rendre.

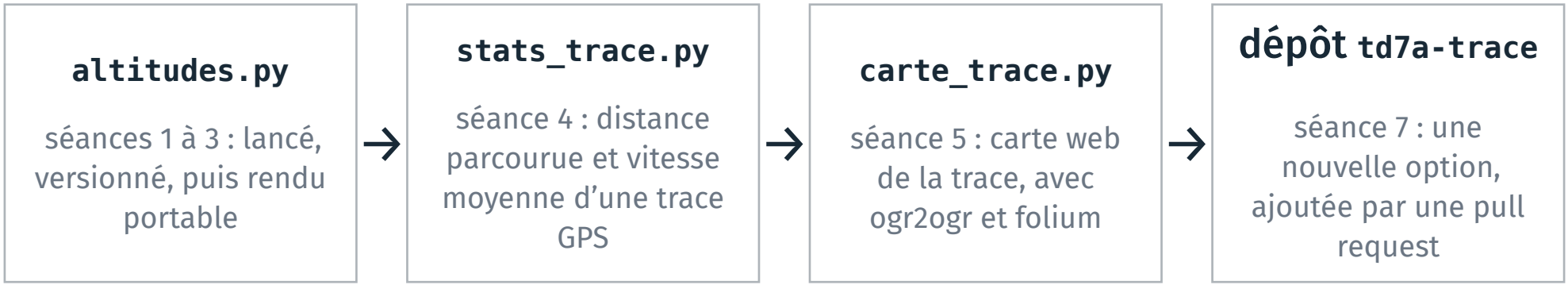
Notions reprises d’une séance à l’autre

Chaque séance reprend la ligne de commande ou git, sur un exercice court.

Séance	Ligne de commande	git
1	commandes de base, chemins	
2	commandes git	dépôt, commit, branche, fusion
3	arguments d’un programme : argparse	fusion d’une branche
4	dossier du projet, options du programme	un commit par étape
5	appel d’un programme depuis Python : subprocess, PATH	
6		dépôt sur GitHub : clone, push, pull
7	option `--profil`	branche, push, pull request

Évolution du programme des TD

Les TD reprennent un même programme, qui calcule des statistiques sur des altitudes, puis sur une trace GPS.



Le TD 6a, sur GitHub, emploie son propre dépôt.

Pratique après le module

Le module introduit ces notions sans les couvrir en entier. Elles s'acquièrent en les pratiquant, dès qu'un cours ou un projet demande d'écrire du code.

Autres cours

TD et projets en Python : terminal, environnement, fichiers

Projets à plusieurs

un dépôt git partagé, des branches, des pull requests

Seul

jeux et exercices de la page « S'entraîner » du site

Cours 1

Logiciels, fichiers, terminal et premier programme

1re année géomatique
2027-2028

Objectifs du cours 1

Rappeler	Découvrir	Exécuter
les bases du fonctionnement d'un ordinateur : composants, logiciels, fichiers, stockage	la ligne de commande : le terminal, les commandes de base, les chemins	un premier programme Python, dans le terminal puis dans un notebook

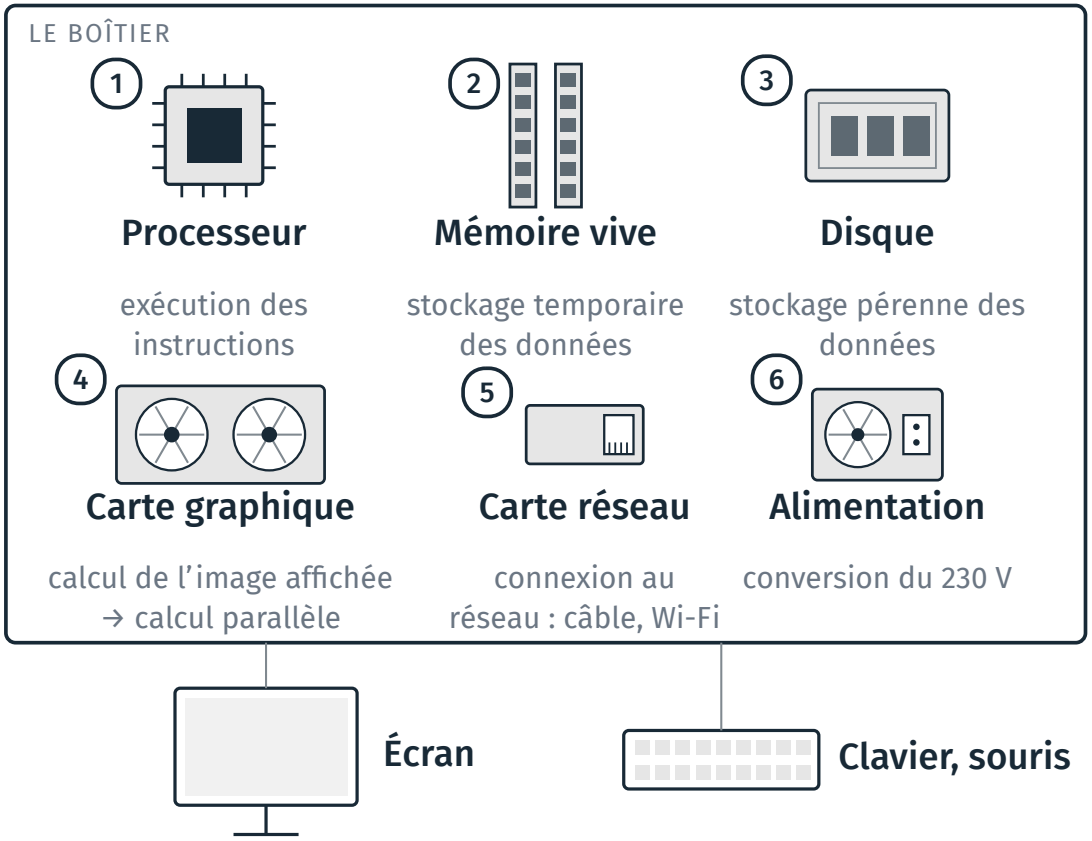
L'écriture de programmes relève du cours de programmation.

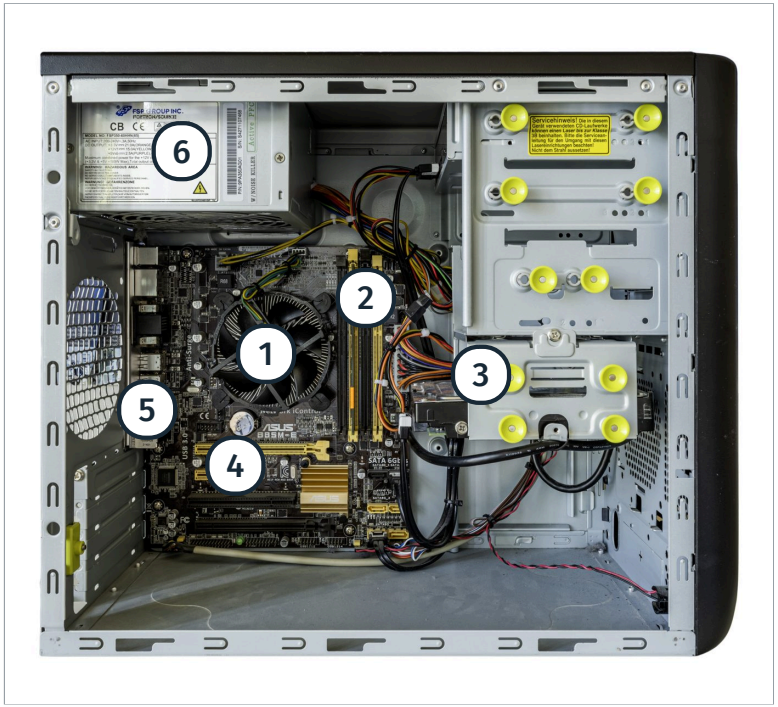
> **Attention.** Une partie de la séance 1 reprend des notions surement vues au lycée et/ou prépa, et avance vite : posez vos questions tout de suite.

Matériel, logiciels, fichiers et stockage

Quelques définitions et rappels

Les composants d’un ordinateur





Un PC de bureau des années 2010, panneau retiré. Photo PantheraLeo1359531, Wikimedia Commons, CC BY 4.0.

Température du processeur

> ? Quelle température atteint un processeur pendant un calcul intensif de plusieurs minutes ?

40 °C	60 °C	90 °C	150 °C

Température du processeur — réponses

Pendant un calcul intensif, un processeur travaille entre 60 et 100 °C.

Situation	Température de la puce
au repos, bureautique	30 à 50 °C
calcul intensif, poste de bureau	60 à 90 °C
calcul intensif, portable fin	80 à 100 °C
limite du fabricant, où le processeur ralentit	95 °C (AMD), 100 °C (Intel)

Ordres de grandeur ; limites des Ryzen 7000 et des Core de 13e génération, données par les fabricants.

Consommation électrique des appareils

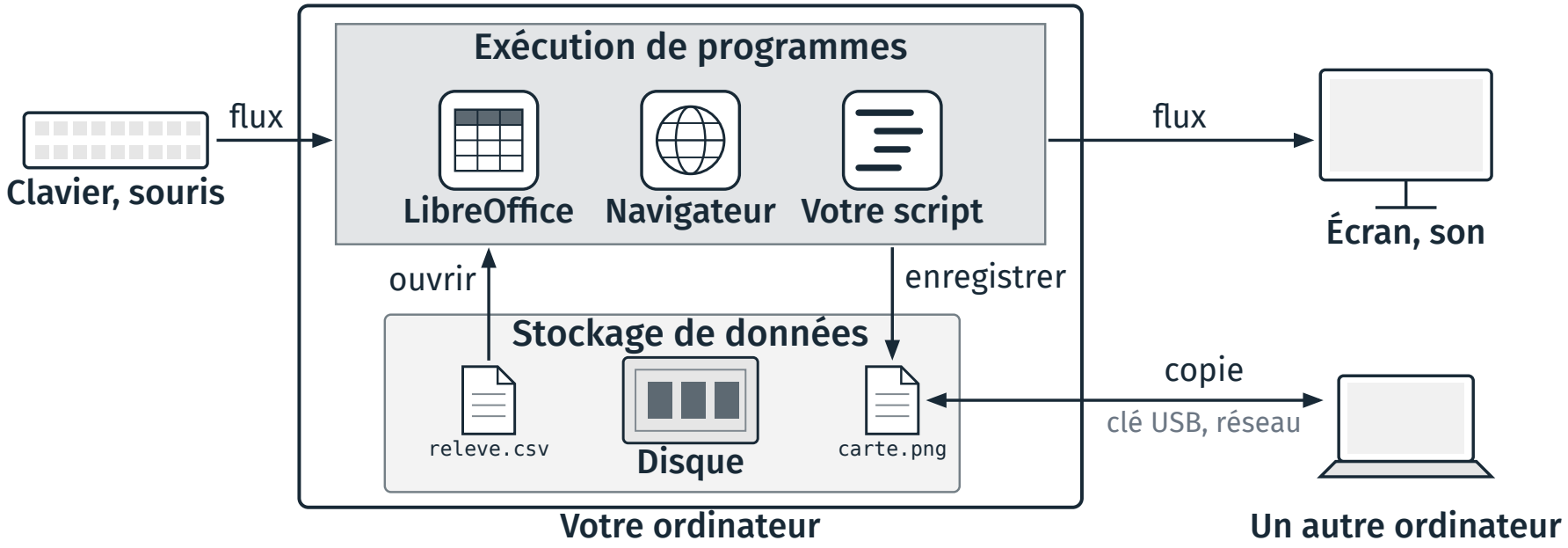
Un serveur GPU consomme en quelques heures l'électricité d'un portable pendant un an.

Appareil	Puissance	Électricité par an
téléphone à touches ou smartphone	0,01 à 5 W	0,1 à 7 kWh
ampoule LED	5 à 10 W	7 à 15 kWh
ordinateur portable	15 à 45 W	30 à 60 kWh
réfrigérateur	20 à 40 W en moyenne	170 à 350 kWh
PC de bureau et carte graphique	100 à 400 W	200 à 400 kWh
logement, tous appareils compris	500 W en moyenne	4 300 kWh
serveur, en continu	300 à 800 W	3 000 à 7 000 kWh
serveur GPU pour l'IA générative, en continu	10 000 W	90 000 kWh

Ordres de grandeur ; portable 8 h par jour, ampoule 4 h par jour. En violet, les repères domestiques (ADEME, Enedis 2024). GPU : carte graphique, *graphics processing unit*. Serveur GPU : NVIDIA DGX H100, à pleine puissance toute l'année.

Exécution de programmes et stockage de données

Deux fonctions principales d'un ordinateur sont l'exécution de programmes et le stockage de données.



Exécution : couches matérielles et logicielles

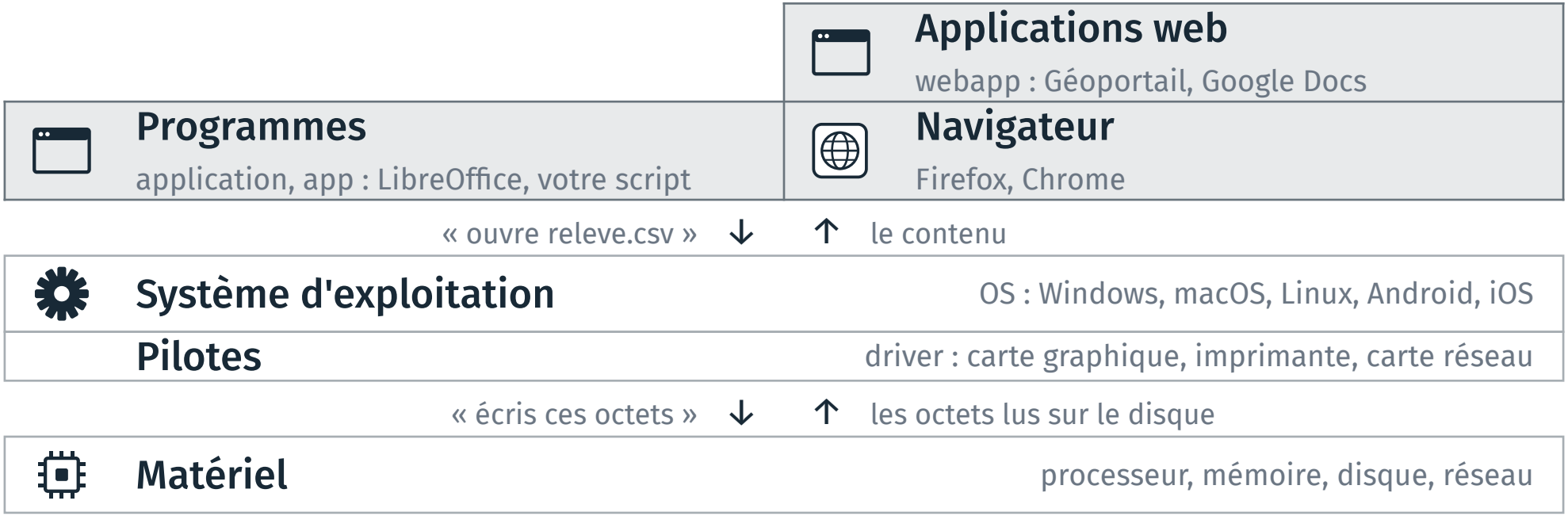
Un programme ne s'adresse pas directement au matériel : il passe par le système (OS).



> Où placer une webapp et un driver sur ce schéma ? Quel OS tourne sur votre téléphone ?

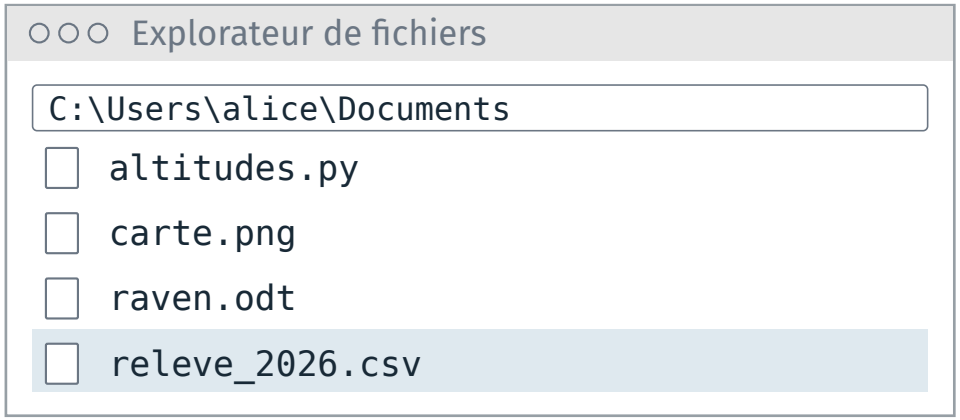
Exécution : couches matérielles et logicielles — réponses

Un programme ne s'adresse pas directement au matériel : il passe par le système (OS).



Stockage : fichier, extension et type

Sur un disque, les données sont stockées dans des fichiers, rangés dans des dossiers. Un fichier permet de transférer des données entre logiciels et entre machines.



L'explorateur affiche les fichiers du dossier Documents.

Le type d'un fichier (texte, vidéo…) est indiqué par son **extension**. Le système s'en sert pour choisir le logiciel à lancer.

releve_2026**.csv**

le nom, que vous choisissez — l'extension

Attention. Windows masque les extensions qu'il connaît : `raven.odt` s'affiche `raven`. Réglage à changer une fois, avant le TD 1a.

Stockage : formats et extensions

Pour chacune de ces extensions, dites de quel type de contenu il s'agit, et si le fichier est lisible dans un éditeur de texte.

`.mp3`	`.mp4`	`.jpg`	`.png`
`.svg`	`.tif`	`.pdf`	`.odt`
`.xlsx`	`.csv`	`.zip`	`.exe`
`.py`	`.md`	`.json`	`.yaml`

Stockage : formats et extensions — réponses

`.mp3` son, avec perte	`.mp4` vidéo, la plus courante	`.jpg` photo, avec perte	`.png` image, sans perte
`.svg` image vectorielle : texte	`.tif` image, y compris GeoTIFF	`.pdf` document mis en page	`.odt` LibreOffice, archive ZIP
`.xlsx` Excel, archive ZIP	`.csv` tableau : du texte	`.zip` archive de fichiers	`.exe` programme Windows
`.py` code Python : du texte	`.md` documentation : du texte	`.json` données, réglages : texte	`.yaml` réglages : du texte

Six de ces seize formats sont du texte : ceux qu'on peut ouvrir dans un éditeur, comparer ligne à ligne et versionner. En bleu, les quatre que vous écrirez vous-mêmes dans ce module.

Quizz : vocabulaire associé aux chemins de fichier

Un chemin dit où trouver un fichier dans l'arborescence des dossiers. Les parties d'un chemin ont chacune un nom : dites à quoi chacun correspond dans cet exemple.

`C:\Users\alice\Documents\raven.odt`

Nom de la partie du chemin	Ce qu'il désigne dans l'exemple
la racine, ou le disque	
un nom de dossier	
le nom du fichier	
le chemin du fichier	
le dossier parent	

Quizz : vocabulaire associé aux chemins de fichier - Réponse

`C:\Users\alice\Documents\raven.odt`

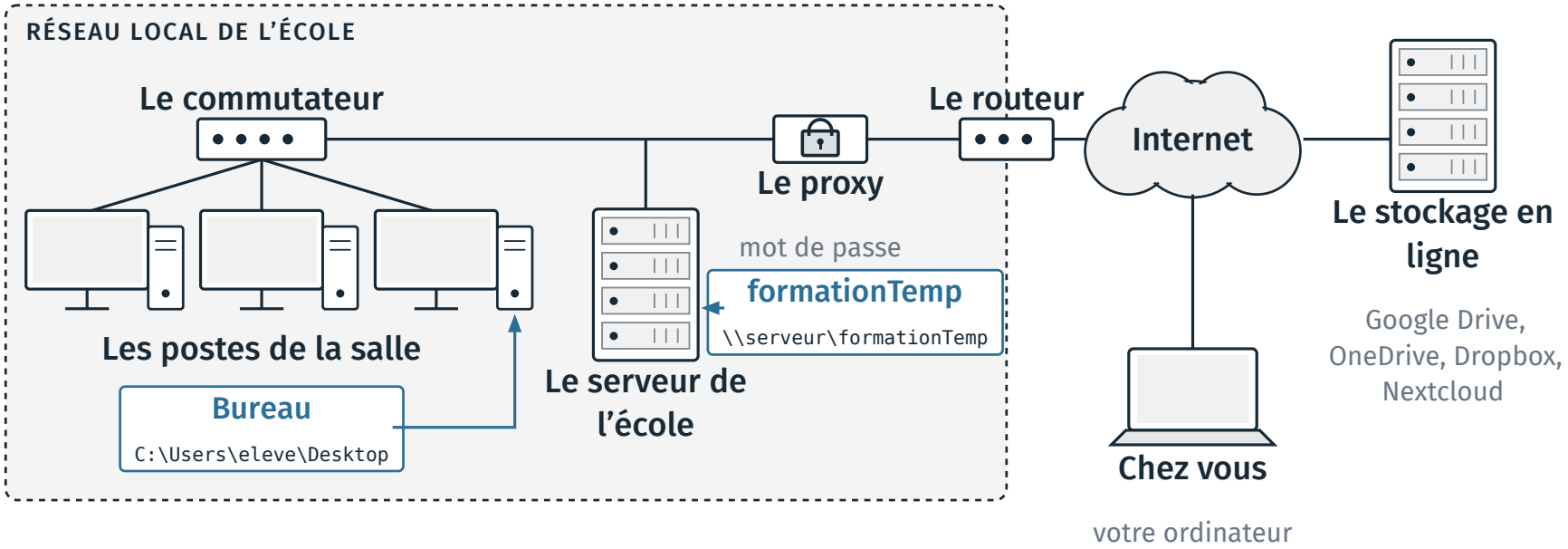
la racine, ou le disque — trois noms de dossier — le nom du fichier, extension comprise

Nom de la partie du chemin	Ce qu'il désigne dans l'exemple
la racine, ou le disque	`C:\` ou `D:\` sous Windows ; `/` sous macOS et Linux
un nom de dossier	`Users`, `alice`, `Documents`
le nom du fichier	`raven.odt`, extension comprise
le chemin du fichier	tout, de la racine au fichier
le dossier parent	`C:\Users\alice\Documents`, le dossier qui le contient

Un chemin se lit de gauche à droite, de la racine au fichier ; chaque séparateur descend d'un dossier.

Les emplacements de stockage sur le réseau

Le Bureau est sur le disque de l'ordinateur, `formationTemp` sur le disque du serveur de l'école. Pour aller sur Internet, les postes passent par le proxy de l'école, avec demande de mot de passe.



Un chemin qui commence par `\\` désigne un dossier d'une autre machine, ici le serveur.

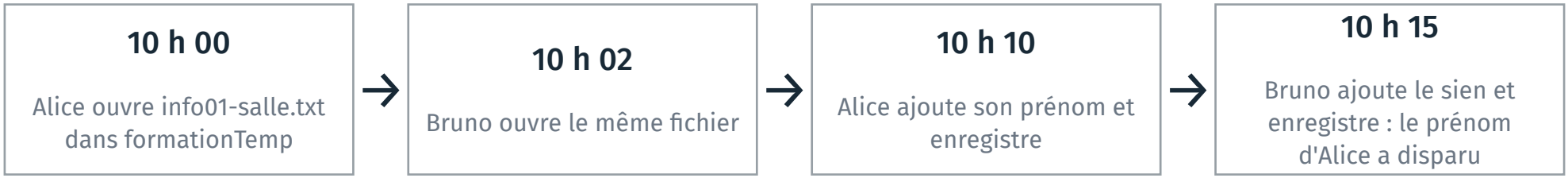
TRAVAUX DIRIGÉS

TD 1a Fichiers, formats et extensions ≈ 15′

`cours1/1a_formats/`

Modifications simultanées d’un fichier partagé

Un fichier du dossier partagé est le même pour toute la salle. Avec un éditeur de texte, le dernier qui enregistre remplace la version de l’autre.



Attention. Ne pas travailler dans `formationTemp`. Copier les fichiers sur le poste, et travailler dans la copie.

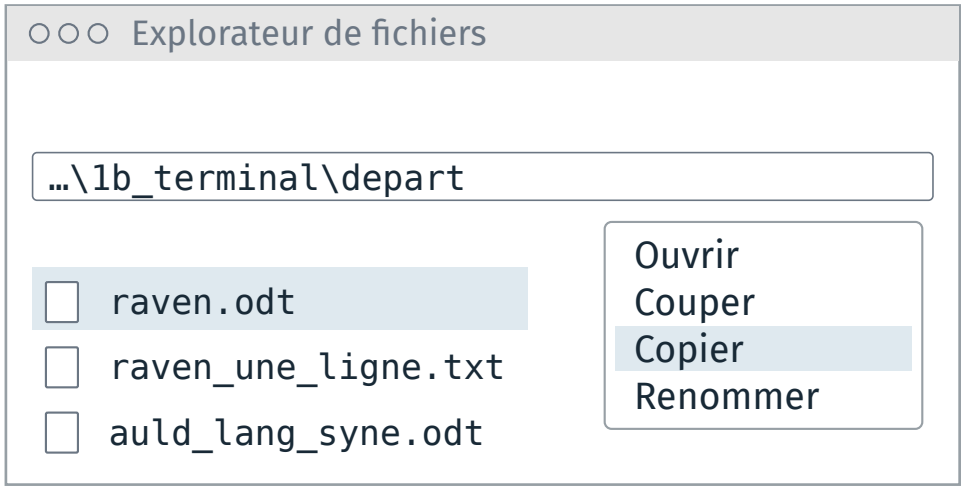
Interface en ligne de commande / terminal

Se déplacer dans les dossiers et agir sur les fichiers en tapant des commandes.

Interface graphique et ligne de commande

Dans une interface graphique, on choisit l'opération parmi les menus et boutons affichés. En ligne de commande, on tape une commande, le nom de l'opération, puis ses paramètres.

Interface graphique : l'explorateur de fichiers



La copie demande ensuite d'ouvrir `travail`, puis de choisir Coller au clic droit.

Ligne de commande : Git Bash



On tape la ligne, puis on appuie sur Entrée. `cp` abrège *copy*.

Intérêt de la ligne de commande

Intérêt face à une interface graphique

Automatisation des tâches

Le traitement par lot, comme cette copie des `.txt` de `depart`, est plus courant qu'en interface graphique. Une suite de commandes s'enregistre dans un script, et se relance avec d'autres paramètres.

```
$ cp depart/*.txt travail/textes/
```

Programmes plus simples à écrire

Le cours 3 construit ce programme. Avec une interface graphique, il faudrait aussi programmer les fenêtres, les boutons et leur comportement.

```
$ python altitudes.py trace.csv -s 2
```

Intérêt de savoir s'en servir

Logiciels scientifiques et serveurs de calcul

Beaucoup de ces outils ne s'utilisent qu'en ligne de commande : GDAL pour les données géographiques, PDAL pour les nuages de points, et les serveurs de calcul, utilisés à distance.

```
$ gdal_translate carte.png carte.tif
```

Le terminal et l'interpréteur de commandes

Le terminal est la fenêtre : il affiche du texte et transmet ce qu'on tape. L'interpréteur de commandes reçoit chaque ligne et lance le programme qu'elle nomme.



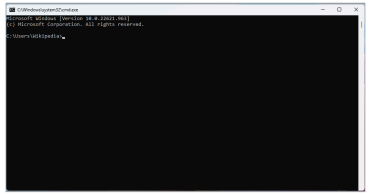
Git Bash

```
eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1/1b_terminal
$ ls depart
auld_lang_syne.odt                raven.odt
auld_lang_syne_une_ligne.donnees  raven_une_ligne.donnees
auld_lang_syne_une_ligne.txt      raven_une_ligne.txt

eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1/1b_terminal
$
```

Le mot « terminal » désigne souvent les deux à la fois. Le langage des commandes est celui de l'interpréteur : cmd, PowerShell ou bash.

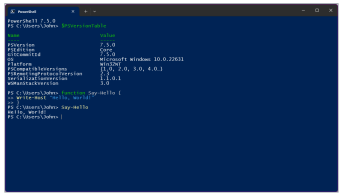
Les trois terminaux du poste de l’école



Invite de commandes (cmd)

Démarrer → `cmd`

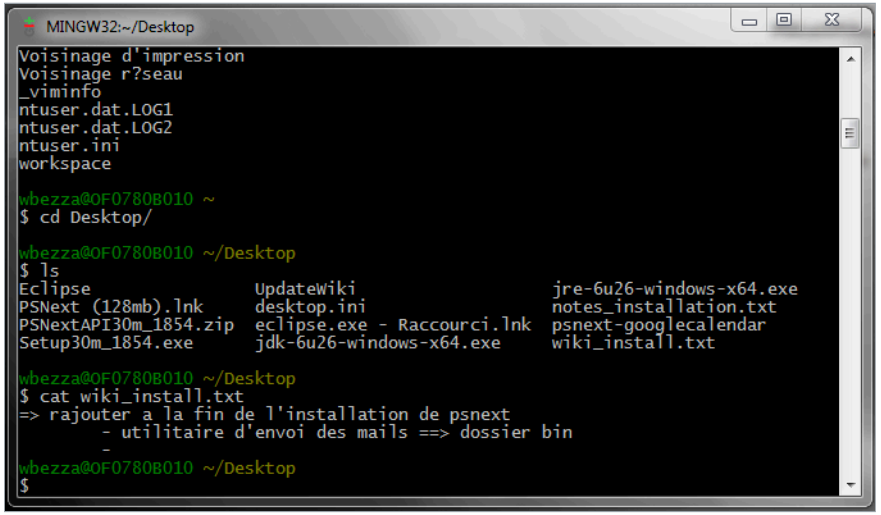
Invite : `C:\Users\…>`



PowerShell

Démarrer → Terminal

Invite : `PS C:\Users\…>`



bash / Git Bash

clic droit → Open Git Bash here

Invite : `utilisateur@machine`, puis `$`

> **Le module emploie Git Bash.** Git Bash est installé avec git sur les postes. Son interpréteur, bash, est celui de Linux et de la plupart des serveurs.

L’invite de Git Bash

Avant chaque commande, Git Bash affiche une invite qui rappelle l’utilisateur, la machine et le dossier courant. La commande se tape après le `$`.

MINGW64:/c/Users/eleve/Desktop/info01/cours1

```
eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1
$ ls
1a_formats/ 1b_terminal/ 1c_programme/ 1d_notebook/

eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1
$
```

Dans l’invite	Ce qu’il désigne
`eleve`	l’utilisateur
`POSTE-12`	la machine
`MINGW64`	Git Bash sous Windows
`~`	le dossier personnel, `C:\Users\eleve`
`~/Desktop/…`	le dossier courant
`$`	l’attente d’une commande

Le `/` à la fin d’un nom signale un dossier. Après le TD 1c, une ligne `(base)` s’ajoute au-dessus : l’environnement conda actif.

Structure d’une commande

Une commande commence par le nom du programme à lancer. Suivent des options, qui commencent par un tiret, et des arguments, souvent des noms de fichiers.

Commande tapée	Programme	Options	Arguments
`ls`	`ls`		
`ls -a depart`	`ls`	`-a`	`depart`
`cp depart/raven.odt travail/`	`cp`		`depart/raven.odt`, `travail/`
`python --version`	`python`	`--version`	
`ls --help`	`ls`	`--help`	

Une option d’une lettre prend un tiret, une option en toutes lettres en prend deux. `--help` affiche l’aide de la plupart des commandes.

Les commandes de base : se déplacer

Chaque commande de base a son équivalent à la souris dans l'explorateur de fichiers.

Commande	Ce qu'elle fait	Dans l'explorateur
`pwd`	affiche le dossier courant	la barre d'adresse
`ls`	liste le dossier courant	la fenêtre ouverte
`ls dossier`	liste le dossier nommé	ouvrir ce dossier
`cd dossier`	descend dans le dossier nommé	double-clic sur le dossier

Les commandes de base : agir sur les fichiers

Commande	Ce qu'elle fait	Dans l'explorateur
`cp source destination`	copie un fichier	`Ctrl` + `C`, puis `Ctrl` + `V`
`mv source destination`	déplace ou renomme	glisser ; `F2`
`mkdir nom`	crée un dossier	Nouveau dossier
`rm fichier`	supprime, sans passer par la corbeille	`Suppr`
`start fichier`	ouvre avec le logiciel associé	double-clic

Le chemin d'un fichier en ligne de commande

En ligne de commande, un fichier se désigne par son chemin, écrit après le nom de la commande. Un chemin **absolu** part de la racine, un chemin **relatif** du dossier où l'on se trouve.

Dans l'explorateur	Dans Git Bash	
`C:\Users\eleve`	`/c/Users/eleve`	chemin absolu
`C:\Users\eleve`	`~`	le dossier personnel
le dossier au-dessus	`..`	le dossier parent

```
~/Desktop/info01/cours1/
├─ 1a_formats/depart/raven.odt
└─ 1c_programme/depart/
altitudes.py
```

	Le chemin de `raven.odt`	Depuis
Absolu	`/c/Users/eleve/Desktop/info01/cours1/1a_formats/depart/raven.odt`	la racine
Relatif	`1a_formats/depart/raven.odt`	`cours1`
Relatif qui remonte	`../1a_formats/depart/raven.odt`	`1c_programme`

Exemples dans Git Bash, qui écrit les chemins à la façon de Linux et de macOS : le disque `C:` devient `/c`, et chaque `\` devient `/`. Sous Windows, Python accepte aussi les `/`.

Le fichier exécutable d'une commande

Une commande lance un fichier exécutable. `which` affiche le chemin de ce fichier, et ce chemin peut remplacer le nom de la commande.

Git Bash

```
$ which notepad
/c/Windows/system32/notepad
$ /c/Windows/system32/notepad.exe depart/raven_une_ligne.txt
```

Commande tapée	Fichier lancé
`notepad fichier`	le premier fichier `notepad` trouvé dans les dossiers du `PATH`, ici `/c/Windows/system32/notepad`
`/c/Windows/system32/notepad.exe fichier`	le fichier de ce chemin, `C:\Windows\System32\notepad.exe`, sans recherche

Les deux commandes ouvrent le fichier dans le Bloc-notes. Quand plusieurs versions d'un programme sont installées sur le poste, comme Python au TD 1c, il faut écrire le chemin du fichier, ou régler le `PATH`.

Fichiers cachés et caractère générique *

```
1b_terminal/
├── depart/
│   ├── auld_lang_syne.odt
│   ├── auld_lang_syne_une_ligne.donnees
│   ├── auld_lang_syne_une_ligne.txt
│   ├── raven.odt
│   ├── raven_une_ligne.donnees
│   └── raven_une_ligne.txt
└── travail/            vide
```

Le dossier du TD 1b, tel qu'il est livré.

Un nom qui commence par un point désigne un fichier caché : `ls` ne l'affiche pas, `ls -a` l'affiche.

```
$ ls travail
$ ls -a travail
./  ../
```

`.` et `..`, le dossier courant et son parent, sont les deux entrées cachées de tout dossier.

Dans un nom de fichier, le caractère générique `*` remplace n'importe quelle suite de caractères.

Commande	Noms désignés dans `depart/`
`ls depart/*.txt`	les deux fichiers `.txt`
`ls depart/raven*`	les trois fichiers de *The Raven*

TRAVAUX DIRIGÉS

TD 1b Rejouer le TD 1a mais en ligne de commande ≈ 15′

`cours1/1b_terminal/`

Exécuter un programme

Exécuter et ouvrir des fichiers de code Python.

Du code au programme : compilation puis exécution

Le processeur n’exécute que des instructions machine. Un compilateur traduit une fois le texte du programme en instructions machine, et les enregistre dans un fichier exécutable.

Compilé : C, C++

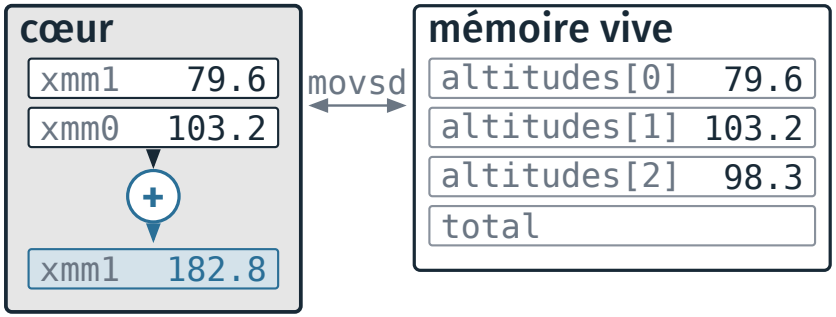


Le fichier exécutable se lance seul, sans le compilateur, sur le système pour lequel il a été compilé.

Instructions machine : une ligne de C++

Une ligne de code devient plusieurs instructions. Le processeur ne calcule que sur ses registres, des cases de mémoire placées dans la puce.

```
double total = altitudes[0] + altitudes[1] + altitudes[2];
moyenne = total / 3;
```



→

```
movsd  xmm1, QWORD PTR altitudes[rip]
movsd  xmm0, QWORD PTR altitudes[rip+8]
addsd  xmm1, xmm0
movsd  xmm0, QWORD PTR altitudes[rip+16]
addsd  xmm0, xmm1
movsd  QWORD PTR -8[rbp], xmm0
```

`movsd a, b`	copie `b` dans `a`, entre la mémoire et un registre
`addsd a, b`	ajoute le registre `b` au registre `a`
`xmm0`, `xmm1`	deux registres du processeur
`altitudes[rip]`	`altitudes[0]`, en mémoire vive
`-8[rbp]`	la variable `total`, en mémoire vive

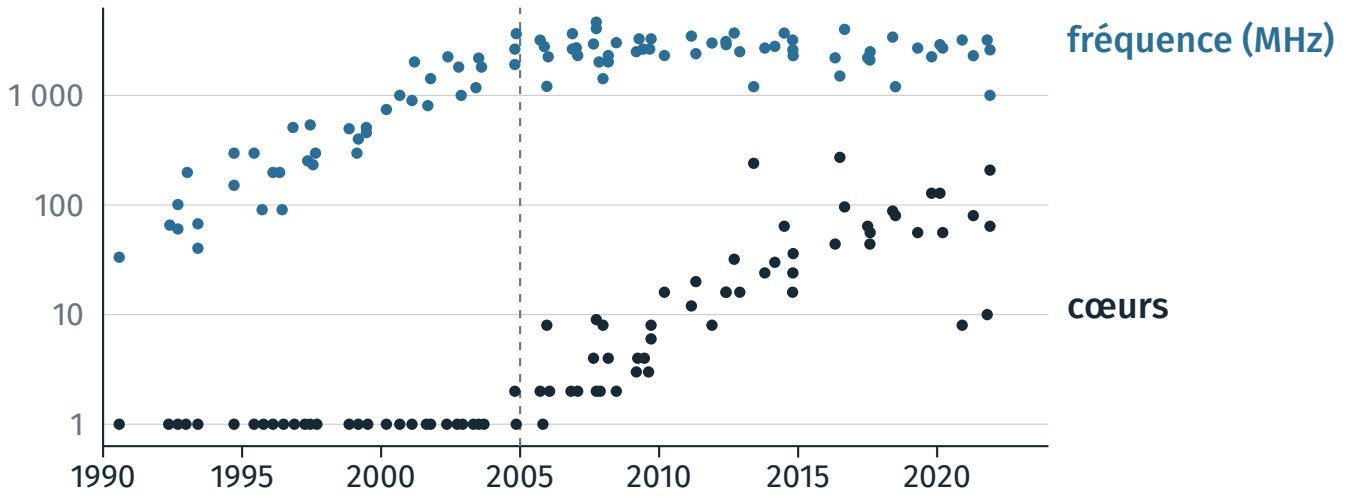
Pendant l'instruction `addsd xmm1, xmm0`, surlignée à droite, le cœur additionne deux registres. Il exécute les instructions une par une, une par cycle.

En bleu, les instructions de calcul. Sortie de g++ sans optimisation (`-O0`), pour un processeur x86-64.

Évolution de la puissance de calcul

Un processeur exécute plus d'instructions par seconde si sa fréquence monte ou s'il a plus de cœurs. Depuis 2005, la fréquence ne monte plus, et le nombre de cœurs augmente.

$2 \cdot 10^9$
cycles par seconde
×
6
cœurs
=
$12 \cdot 10^9$
opérations par seconde



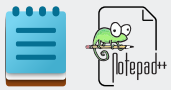
Calcul pour l'AMD Ryzen 5 7530U (2023) à sa cadence de base : 2 GHz, soit 2 milliards de cycles par seconde, ou 2 000 MHz. Un programme Python ordinaire n'occupe qu'un cœur. Un point par processeur, échelle logarithmique ; Karl Rupp, *microprocessor-trend-data*, CC BY 4.0.

Du code au programme : interpréteur

Un interpréteur lit le texte du programme et l'exécute, à chaque lancement, sans fichier exécutable. Python est un langage interprété.

Interprété : Python

`altitudes.py`	→	`python`	→	`moyenne : 93.7 m`
un fichier texte, en entrée		l'interpréteur, un programme en ligne de commande		le résultat, écrit dans le terminal



écrit dans Notepad++ ou le Bloc-notes

lancé dans Git Bash : `python altitudes.py`

rien d'autre sur le disque

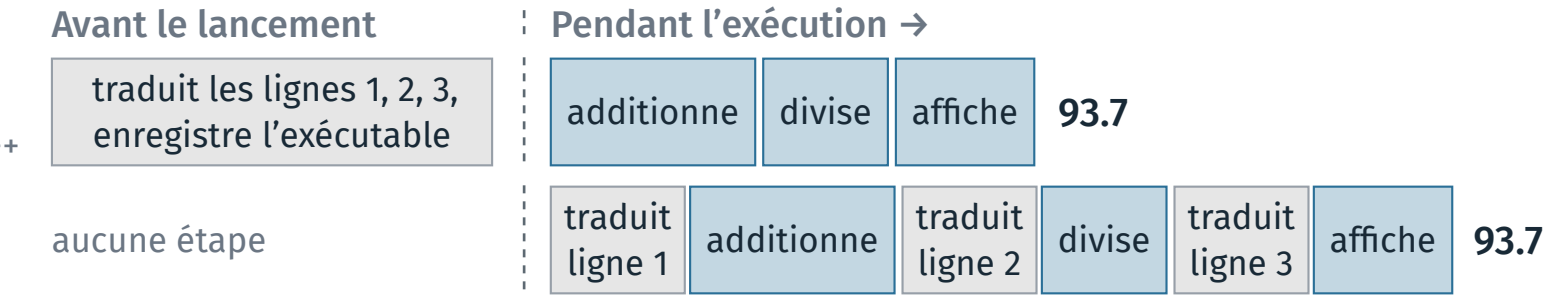
L'interpréteur est lui-même un programme compilé : `python.exe` est écrit en C. Pour lancer `altitudes.py` sur une autre machine, Python doit y être installé.

Compilateur et interpréteur : traduction

Le compilateur traduit toutes les lignes avant le lancement. L'interpréteur lit chaque ligne, la traduit et l'exécute, puis passe à la suivante.

Un programme de trois lignes, en Python

```
1  total = 79.6 + 103.2 + 98.3
2  moyenne = total / 3
3  print(moyenne)
```



Au lancement suivant, l'exécutable se relance sans traduction, et l'interpréteur traduit de nouveau les trois lignes.

Python, un programme en ligne de commande

Dans un terminal, la commande `python` suivie du nom d'un fichier exécute ce fichier. Tapée seule, elle ouvre une session interactive, où Python exécute chaque ligne dès qu'on appuie sur Entrée.

Exécuter un fichier : `python altitudes.py`

Git Bash

```
$ cd ~/Desktop/info01/cours1/1c_programme/depart
$ python altitudes.py
moyenne : 93.7 m
```

Ouvrir une session interactive : `python`

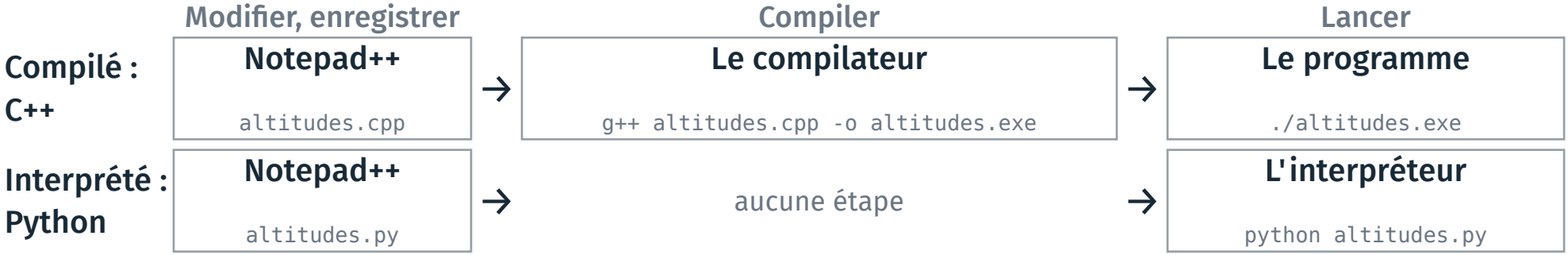
Git Bash

```
$ python
Python 3.12.14 (main, Sep  2 2026, 23:27:36) [GCC
15.3.0] on linux
>>> 79.6 + 103.2
182.8
>>> exit()
```

Python écrit ses résultats dans le terminal, sans ouvrir de fenêtre. Comme `ls` ou `cp`, `python` est une commande, qu'un script peut lancer.

Compilateur et interpréteur : essais, calculs

La commande `python altitudes.py` lance l'interpréteur, qui lit le fichier enregistré et l'exécute. Un programme compilé demande une commande de plus.



Situation	Compilé : C++	Interprété : Python
Modifier `altitudes`, relancer	compiler, puis lancer : 0,06 s	lancer : 0,01 s
Essayer une ligne seule	compiler tout un programme	session interactive, `>>>`
Calcul de 100 millions d'additions	0,08 s, traduit une seule fois	5 s, chaque addition traduite

En bleu, le côté favorable. L'interprétation convient à un calcul fait une fois et à l'exploration, la compilation à un calcul répété souvent.

Compilateur et interpréteur : changer de poste

Un programme Python dépend de l'interpréteur du poste. `which python` affiche celui que lance la commande, et le chemin complet de `python.exe` en lance un autre.

Situation	Compilé : C++	Interprété : Python
Lancer sur un poste sans Python	`altitudes.exe` se lance seul	installer Python
Lancer sur un autre système	recompiler pour ce système	le même `.py`, si Python est installé

Dans Git Bash : le Python par défaut échoue, celui d'Anaconda réussit

Git Bash

```
$ python altitudes.py
SyntaxError: Non-ASCII character '\xc3' in file altitudes.py on line 1…
$ which python
/c/Python27/python
$ /c/ProgramData/anaconda3/python.exe altitudes.py
moyenne : 93.7 m
```

Exemple d'un poste où un Python 2.7 est installé dans `C:\Python27`.

Programmation littérale

Un notebook réunit le texte, le code et son résultat dans un seul document.



Le terme « programmation littérale » est de Donald Knuth, 1984.

TRAVAUX DIRIGÉS

TD 1c Lancer un programme

`cours1/1c_programme/`

≈ 20′

TRAVAUX DIRIGÉS

TD 1d Un notebook dans JupyterLab ≈ 12′

`cours1/1d_notebook/`

À retenir

Récapituler les commandes de la séance et l'édition d'un programme Python.

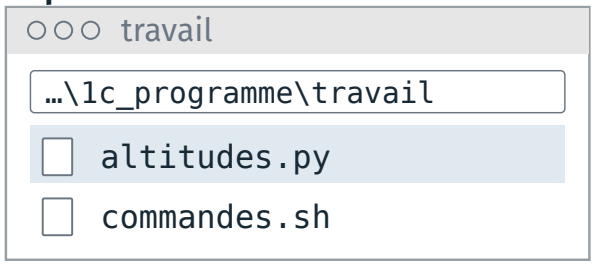
Aide-mémoire : terminal et commandes

Se déplacer, lister		Agir sur les fichiers	
`pwd`	affiche le dossier courant	`cp source destination`	copie un fichier
`ls dossier`	liste le dossier nommé	`mv source destination`	déplace ou renomme
`ls -a dossier`	liste aussi les fichiers cachés	`mkdir nom`	crée un dossier
`cd dossier`	descend dans le dossier nommé	`rm fichier`	supprime, sans corbeille
`cd ..`	remonte au dossier parent	`start fichier`	ouvre avec le logiciel associé
`which commande`	affiche le chemin du fichier lancé	`commande --help`	affiche l'aide de la commande

Chemins dans Git Bash		Noms et raccourcis	
`/c/Users/eleve`	`C:\Users\eleve` dans l'explorateur	`*`	remplace toute suite de caractères
`~`	le dossier personnel	`"le corbeau.txt"`	reste un seul argument, malgré l'espace
`..`	le dossier parent	`Tab`	complète un nom commencé
`.`	le dossier courant	flèche vers le haut	rappelle la commande précédente

Aide-mémoire : programme Python

Explorateur de fichiers



ouvrir →

Éditeur de texte Bloc-notes, Notepad++

altitudes.py - Notepad++

```
1 altitudes = [79.6, 103.2, 98.3]
2 total = 0
3 for altitude in altitudes:
4     total = total + altitude
5 moyenne = total / len(altitudes)
6 print(f"moyenne : {moyenne:.1f} m")
```

pour un programme de plusieurs fichiers

↓ enregistrer, puis lancer

> Un programme écrit dans LibreOffice Writer est enregistré en `.odt`, et `python` ne peut pas exécuter ce fichier.

Terminal Git Bash

Git Bash

```
eleve@POSTE-12 MINGW64 ~/Desktop/info01/cours1/1c_programme/travail
$ python altitudes.py
moyenne : 93.7 m
```

Après chaque modification, enregistrer dans Notepad++, puis relancer dans Git Bash. `python` seul ouvre une session interactive, et `exit()` la quitte. `which python` affiche le Python lancé.